

4. Análisis y diseño de algoritmos:

4.1. ¿Qué son los algoritmos?

“Un algoritmo es una secuencia de pasos lógicos que permiten solucionar un problema, dado un estado inicial y una entrada, para llegar a un estado final y una salida”

Como hemos visto, la robótica es la unión de la electrónica y la programación, y a medida que se nos vayan presentando diferentes problemas, las soluciones se vuelven más complejas, es por eso que es importante crear algoritmos útiles.

Para esto es crucial entender que se tiene una entrada, que es la información que se le da al algoritmo, un proceso, que es lo que realiza la tarea que se quiere con la información dada, y una salida, que es el resultado de nuestro algoritmo.



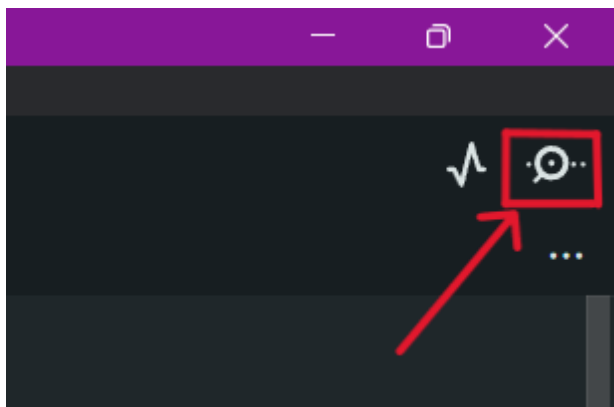
Esto se puede aplicar a muchos contextos más allá de la robótica, y en esta unidad nos vamos a enfocar en programar soluciones a problemas que no necesariamente son de robótica.

4.2. Comunicación Serial

Arduino nos ofrece una forma de comunicarnos con él a través de la pantalla del computador, la cual se conoce como **comunicación serial**.

El correcto uso de esta herramienta nos permitirá tener un mayor control sobre nuestros algoritmos y nos ayudará a detectar errores con mayor facilidad.

Para poder ver los mensajes de nuestro Arduino existe una herramienta dentro del programa llamada consola serial, ubicada en la esquina superior derecha, que básicamente es un chat con nuestro Arduino y sólo es necesario tenerlo conectado mediante USB:



4.2.1. Serial.begin()

El comando `Serial.begin()` es el que se encarga de inicializar la comunicación entre el Arduino y el computador. Debe ir en el `void.setup()`, y dentro de los paréntesis se especifica la velocidad de la conexión (BaudRate), la cual usualmente se establece en 9600.

```
1 void setup() {  
2   //Se inicializa la comunicación serial  
3   Serial.begin(9600);  
4 }
```

¿Sabías qué? *BaudRate* significa tasa de Baudios, y básicamente es el número de señales que se envían por segundo. Va a depender de cada modelo de placa qué velocidades soportan

4.2.2. Serial.print() y Serial.println()

Ya con la conexión establecida, podemos leer información desde el Arduino a través de la consola serial, para esto tenemos los comandos `Serial.print()` y `Serial.println()`, siendo la única diferencia que el último genera un salto de línea. Dentro de los paréntesis se puede poner el texto que queramos entre comillas dobles o simples, o directamente el nombre de una variable. A continuación se puede ver un ejemplo de cómo utilizar estos comandos:

```
1 int edad=15;  
2 Serial.print("Hola");  
3 Serial.println('tengo');  
4 Serial.print(edad);  
5 Serial.println(" años.");
```

Siendo su resultado:

```
Holatengo  
15 años.
```

4.2.3. Serial.available()

Ahora que sabemos cómo leer información desde el Arduino, hay que entender cómo se mandan datos hacia la placa. Esta tiene algo llamado *buffer* que básicamente es una cola donde esperan los caracteres que enviamos a través de la consola serial.

Es por esto que existe un comando para saber si hay datos en la cola, `Serial.available()` que básicamente se puede utilizar como la condición de un `if`, el cual retorna el número de caracteres que hay en la cola.

```

1  if (Serial.available()>0){
2      Serial.println("Hay datos en el buffer");
3  }else{
4      Serial.println("El buffer esta vacío");
5  }

```

4.2.4. Serial.read()

Ya sabemos mostrar la información y si tenemos algo disponible, ahora nos falta poder leer la información y guardarla en una variable, lo cual puede ser logrado con el comando *Serial.read*, que al usarse, debe ser asignado a una variable de tipo *char*, cabe destacar que esta instrucción sólo lee el primer carácter que haya en el buffer, es decir, sólo leerá una letra a la vez.

```

1  char input;
2  if (Serial.available()>0){
3      input = Serial.read()
4      Serial.print("El caracter que leí es: ");
5      Serial.println(input);
6  }

```

4.2.5. Strings y Serial.readStringUntil('\n')

Como pueden suponer, la función anterior puede ser impráctica a la hora de querer leer palabras y/o frases completas de forma rápida y cómoda, por lo cual primero hay que introducir un nuevo tipo de variable, los **String**, que básicamente son variables que almacenan cadenas de caracteres, formando así palabras o frases completas. Funcionan similar a las listas en el sentido que se puede acceder a cada letra de la frase con su índice.

Los strings se pueden crear de la siguiente forma:

```

1  String palabra = "hola mundo";

```

Fijarse que las palabras (o frases) deben estar entre comillas, las cuales pueden ser simples (") o dobles (").

Y se representa de la siguiente manera:

Palabra				
Índice	0	1	2	3
Valor	H	o	l	a

Ahora, para poder leer el string completo desde la consola serial, se necesita el siguiente comando especial: *Serial.readStringUntil('\n')*, el cual sirve para leer muchos caracteres hasta que nosotros le indiquemos, en este caso el *\n*, que representa un salto de línea, el cuál se pone cada vez que apretamos la tecla 'enter'.

Al igual que *Serial.read()*, este comando debe ser asignado a una variable, en este caso, de tipo **String**. Quedando algo así:

```

1  String palabra;
2  if(Serial.available()){
3      palabra = Serial.readStringUntil('\n');
4  }

```

4.3. Debugging

Ahora que tienen todas las herramientas para comunicarse con su placa, existe un concepto en la programación que es el *Debugging*, que básicamente es el proceso para descubrir que es lo que está funcionando mal dentro de nuestro código.

Cuando estemos trabajando en diferentes proyectos de robótica, es probable que no todo funcione correctamente a la primera, es por eso que una técnica básica de *debuggear* es mostrar por pantallas las diferentes variables o mensajes para saber en qué parte del código está el problema, puede ser que nos quedamos atrapados en un bucle sin saber, o que algún cálculo no está bien hecho y no da el resultado esperado, y de esta forma va a ser más fácil descubrir los errores.

4.4. Ejercicios

4.4.1. Preséntate!!

A continuación crea un programa que pregunte por tu nombre y luego lo imprima por la consola serial.

Input:

```

Ingrese su nombre:
> Benjamin

```

Output:

```

Hola benjamin

```

A continuación te mostramos cómo se realiza esto utilizando el último comando que aprendimos, y queda como desafío personal resolverlo sólo con el *Serial.read()*:

```

1  void setup() {
2      //Se inicializa la comunicación serial
3      Serial.begin(9600);
4  }
5
6  void loop() {
7      Serial.println("Ingrese su nombre");
8      //Este while true es para que sólo pregunte una
9      //vez y se quede esperando el input
10     while (true) {
11         if (Serial.available()) {
12             //Se lee la entrada como un string, eso
13             //facilita las cosas
14             String input = Serial.readStringUntil('\n');
15             Serial.print("Hola, ");
16             Serial.println(input);
17             break;
18         }
19     }
20 }

```

4.4.2. Círculos

A continuación crea un programa que pregunte por un radio y devuelva su perímetro y área.

```
Ingrese el radio:  
> 5
```

Output:

```
Perimetro: 31.4  
Area: 78.5
```

El primer paso en el diseño de algoritmos es tener bien claro lo que se pide y ser ordenado al respecto.

En este caso primero tenemos que analizar los datos que tenemos, siendo el radio que es dado por el usuario, y el perímetro y radio, que es lo que nos devuelve el algoritmo como salida, notar que estamos trabajando con decimales, entonces las variables a utilizar son de tipo *float*:

```
1 float radio, area, perimetro;  
2  
3 void setup() {  
4   //Se inicializa la comunicación serial  
5   Serial.begin(9600);  
6 }
```

Luego, hay que pensar en cómo obtener lo que nos piden, y para eso tenemos las fórmulas de perímetro y radio:

$$\text{Perimetro} = 2 * \pi * \text{radio}$$
$$\text{Area} = \pi * \text{radio}^2$$

Lo que traducido en código nos queda así (recordar que $\pi = 3,14$ y $\text{radio}^2 = \text{radio} * \text{radio}$):

```
1 perimetro = 2 * 3.14 * radio  
2 area = 3.14 * radio * radio
```

Y ya con esto podemos armar el resto del código:

```
1  
2 void loop() {  
3   Serial.println("Ingrese el radio");  
4   //Este while true es para que sólo pregunte una  
5   //vez y se quede esperando el input  
6   while (true) {  
7     if (Serial.available()) {  
8       //Se lee la entrada como un string, eso  
9       //facilita las cosas  
10      String input = Serial.readStringUntil('\n');  
11      //Se transforma el string a float  
12      radio = input.toFloat();  
13      //Se hacen los cálculos  
14      perimetro = 2 * 3.14 * radio;  
15      area = 3.14 * pow(radio, 2);  
16      //Se imprime el resultado  
17      Serial.print("El perimetro es: ");  
18      Serial.println(perimetro);  
19      Serial.print("El area es: ");  
20      Serial.println(area);  
21      //El break es para salir del while y volver  
22      //a preguntar el radio  
23      break;  
24    }  
25  }  
26 }
```

Más herramientas Como los *Strings* son palabras, no pueden ser utilizadas en operaciones matemáticas, es por eso que es necesario utilizar la función *.toFloat()* para hacer la transformación.

Más herramientas Arduino ofrece una variedad inmensa de funciones matemáticas, entre ellas *pow()*, el cual toma el primer valor y lo eleva en base al segundo

4.5. Patrones

Si se fijaron, en los 2 ejercicios que hicimos, hay elementos que se repiten e incluso se programan casi igual. Esto es lo que denominamos patrones, y es importante que sean capaces de descubrirlos cuando enfrenten un problema, ya que quizás alguna parte de este ya lo resolvieron anteriormente.

4.6. Desafíos

Ahora te invitamos a intentar resolver los siguientes problemas:

4.6.1. Triángulos

Los tres lados a , b y c de un triángulo deben satisfacer la desigualdad triangular: cada uno de los lados no puede ser más largo que la suma de los otros dos.

$$a < (b + c)$$

$$b < (a + c)$$

$$c < (a + b)$$

Escriba un programa que reciba como entrada los tres lados de un triángulo, e indique:

- Si el triángulo es inválido.
- Si no lo es, qué tipo de triángulo es.

Ejemplo 1:

Input:

```
Ingrese a:
>3.9
Ingrese b:
>6.0
Ingrese c:
>1.2
```

Output:

```
No es un triangulo valido.
```

Ejemplo 2:

Input:

```
Ingrese a:
>1.9
Ingrese b:
>2
Ingrese c:
>2
```

Output:

```
El triángulo es isosceles.
```

Ejemplo 3:

Input:

```
Ingrese a:
>3
Ingrese b:
>5
Ingrese c:
>4
```

Output:

```
El triángulo es escaleno.
```

4.6.2. Secuencia de Collatz

La secuencia de Collatz de un número entero se construye de la siguiente forma:

- si el número es par, se lo divide por dos.
- si es impar, se le multiplica tres y se le suma uno.
- la sucesión termina al llegar a uno.

La conjetura de Collatz afirma que, al partir desde cualquier número, la secuencia siempre llegará a 1. A pesar de ser una afirmación a simple vista muy simple, no se ha podido demostrar si es cierta o no.

Usando computadores, se ha verificado que la sucesión efectivamente llega a 1 partiendo desde cualquier número natural menor que 2^{25}

Desarrolle un programa que entregue la secuencia de Collatz de un número entero.

Ejemplo 1:

Input:

```
n:
>18
```

Output:

```
18 9 28 14 7 22 11 34 17 52 26 13 40 20 10 5
16 8 4 2 1
```

Ejemplo 2:

Input:

```
n:
>19
```

Output:

```
19 58 29 88 44 22 11 34 17 52 26 13 40 20 10 5
16 8 4 2 1
```

Ejemplo 3:

Input:

```
n:
>20
```

Output:

```
20 10 5 16 8 4 2 1
```

Más herramientas Además de los típicos operadores matemáticos (+, -, *, /) existe uno muy útil: %, el cuál da como resultado el resto de una división, por ejemplo:

$$4 \% 2 = 0$$

$$3 \% 2 = 1$$

4.6.3. No múltiplos

Escriba un programa que muestre los números naturales menores o iguales que un número n determinado, que no sean múltiplos ni de 3 ni de 7.

Input:

```
Ingrese numero:  
>24
```

Output:

```
1  
2  
4  
5  
8  
10  
11  
13  
16  
17  
19  
20  
22  
23
```

4.6.4. Alzas del dólar

Un analista financiero lleva un registro del precio del dólar día a día, y desea saber cuál fue la mayor de las alzas en el precio diario a lo largo de ese período.

Escriba un programa que pida al usuario ingresar el número n de días, y luego el precio del dólar para cada uno de los n días.

El programa debe entregar como salida cuál fue la mayor de las alzas de un día para el otro.

Si en ningún día el precio subió, la salida debe decir: **No hubo alzas.**

Input:

```
Cuantos dias?  
>10  
Dia 1:  
>496.96  
Dia 2:  
>499.03  
Dia 3:  
>496.03  
Dia 4:  
>493.27  
Dia 5:  
>488.82  
Dia 6:  
>492.16  
Dia 7:  
>490.32  
Dia 8:  
>490.67  
Dia 9:  
>490.89  
Dia 10:  
>494.10
```

Output:

```
La mayor alza fue de $3.34
```

4.6.5. Promoción con descuento (Difícil)

El supermercado Musta Market ha lanzado una promoción para todos sus clientes. La promoción consiste en aplicar un descuento por cada n productos que pasan por caja.

El primer descuento es de 20 %, y se aplica sobre los primeros n productos ingresados. Luego, cada descuento es la mitad del anterior, y es aplicado sobre los siguientes n productos.

Por ejemplo, si $n = 3$ y la compra es de 11 productos, entonces los tres primeros tienen 20 % de descuento, los tres siguientes 10 %, los tres siguientes 5 %, y los dos últimos no tienen descuento.

Escriba un programa que pida al usuario ingresar n y la cantidad de productos, y luego los precios de cada producto. Al final, el programa debe entregar el precio total, el descuento total y el precio final después de aplicar el descuento.

Input:

```
n:  
>3  
Cantidad productos:  
>8  
Precio producto 1:  
>400  
Precio producto 2:  
>800  
Precio producto 3:  
>500  
Precio producto 4:  
>100  
Precio producto 5:  
>400  
Precio producto 6:  
>300  
Precio producto 7:  
>200  
Precio producto 8:  
>500
```

Output:

```
Total: 3200  
Descuento: 455  
Por pagar: 2745
```

4.6.6. Intersección de círculos(Difícil)

Una circunferencia en el plano está definida por tres valores: las coordenadas (x, y) de su centro, y su radio r .

Escriba un programa que determine si dos circunferencias se intersectan o no.

Input:

```
x1:  
>5  
y1:  
>6  
r1:  
>3.5  
x2:  
>10  
y2:  
>5  
r2:  
>3
```

Output:

```
Se intersectan
```

Input:

```
x1:  
>3.5  
y1:  
>5  
r1:  
>2  
x2:  
>10  
y2:  
>4  
r2:  
>3
```

Output:

```
No se intersectan
```